

# Lab 1 – R Tutorial – First Part

## What is R? – An Introduction

R is a programming language and software environment for statistical analysis, graphics representation and reporting. R was created by Ross Ihaka and Robert Gentleman at the University of Auckland, New Zealand, and is currently developed by the R Development Core Team. R is freely available under the GNU General Public License, and pre-compiled binary versions are provided for various operating systems like Linux, Windows and Mac. This programming language was named **R**, based on the first letter of first name of the two R authors (Robert Gentleman and Ross Ihaka), and partly a play on the name of the Bell Labs Language **S**.

## R – An Overview

### Evolution of R

R was initially written by **Ross Ihaka** and **Robert Gentleman** at the Department of Statistics of the University of Auckland in Auckland, New Zealand. R made its first appearance in 1993.

A large group of individuals has contributed to R by sending code and bug reports.

- Since mid-1997 there has been a core group (the "R Core Team") who can modify the R source code archive.

### Features of R

As stated earlier, R is a programming language and software environment for statistical analysis, graphics representation and reporting. The following are the important features of R:

- R is a well-developed, simple and effective programming language which includes conditionals, loops, user defined recursive functions and input and output facilities.
- R has an effective data handling and storage facility,
- R provides a suite of operators for calculations on arrays, lists, vectors and matrices.
- R provides a large, coherent and integrated collection of tools for data analysis.
- R provides graphical facilities for data analysis and display either directly at the computer or printing at the papers.

As a conclusion, R is world's most widely used statistics programming language. It's the # 1 choice of data scientists and supported by a vibrant and talented community of contributors. R is taught in universities and deployed in mission critical business applications. This tutorial will teach you R programming along with suitable examples in simple and easy steps.

## Why R?

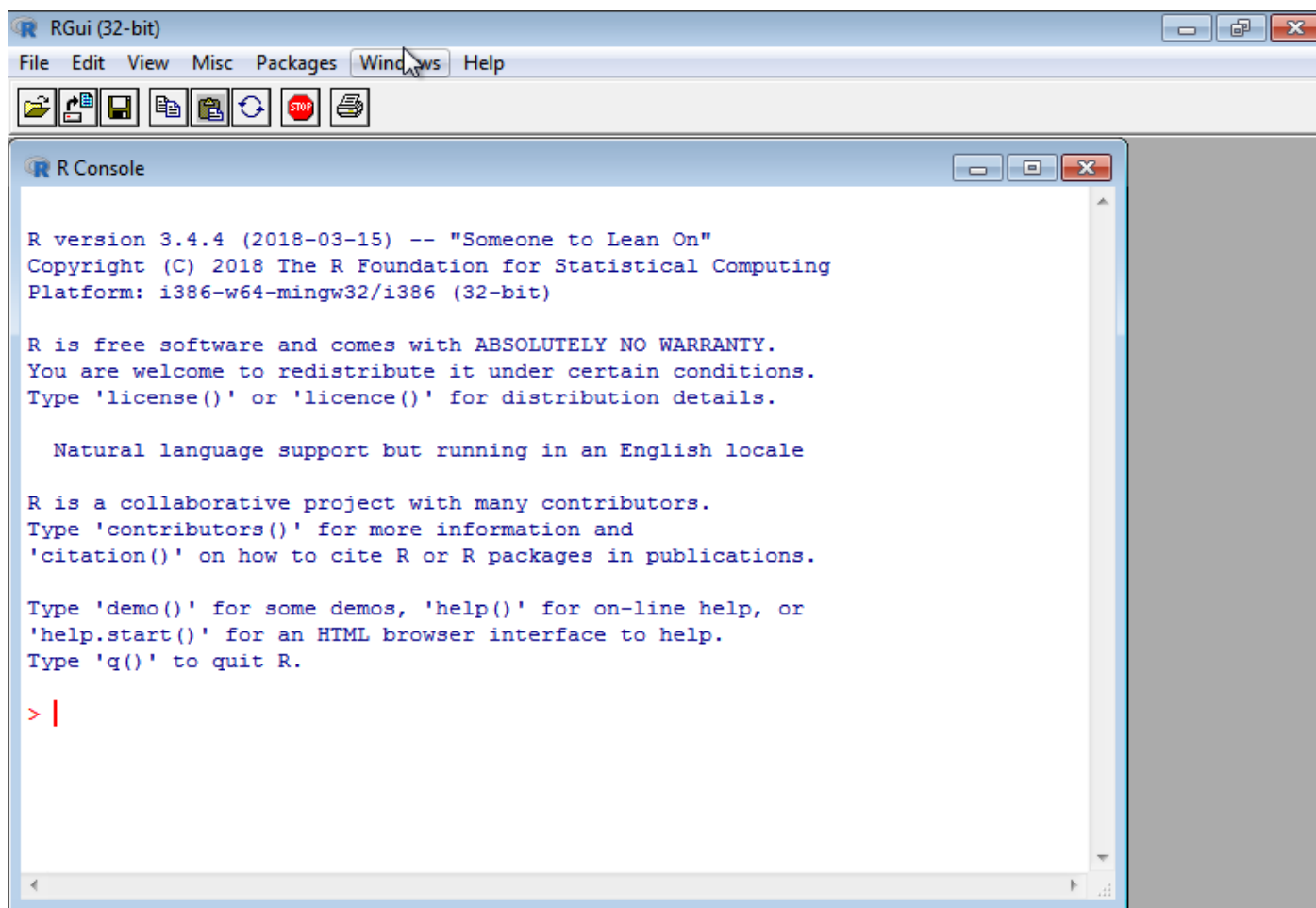
R is not a programming language like C or Java. It was not created by software engineers for software development. Instead, it was developed by statisticians as an interactive environment for data analysis. You can read the full history in the paper [A Brief History of S<sup>5</sup>](#). The interactivity is an indispensable feature in data science because, as you will soon learn, the ability to quickly explore data is a necessity for success in this field. However, like in other programming languages, you can save your work as scripts that can be easily executed at any moment. These scripts serve as a record of the analysis you performed, a key feature that facilitates reproducible work. If you are an expert programmer, you should not expect R to follow the conventions you are used to since you will be disappointed. If you are patient, you will come to appreciate the unequal power of R when it comes to data analysis and, specifically, data visualization.

Other attractive features of R are:

1. R is free and open source.
2. It runs on all major platforms: Windows, Mac Os, UNIX/Linux.
3. Scripts and data objects can be shared seamlessly across platforms.
4. There is a large, growing, and active community of R users and, as a result, there are numerous resources for learning and asking questions
5. It is easy for others to contribute add-ons which enables developers to share software implementations of new data science methodologies. This gives R users early access to the latest methods and to tools which are developed for a wide variety of disciplines, including ecology, molecular biology, social sciences, and geography, just to name a few examples.

## The R console

Interactive data analysis usually occurs on the *R console* that executes commands as you type them. There are several ways to gain access to an R console. One way is to simply start R on your computer. The console looks something like this:



As a quick example, try using the console to calculate a 15% tip on a meal that cost 19.71:

```
0.15 * 19.71
```

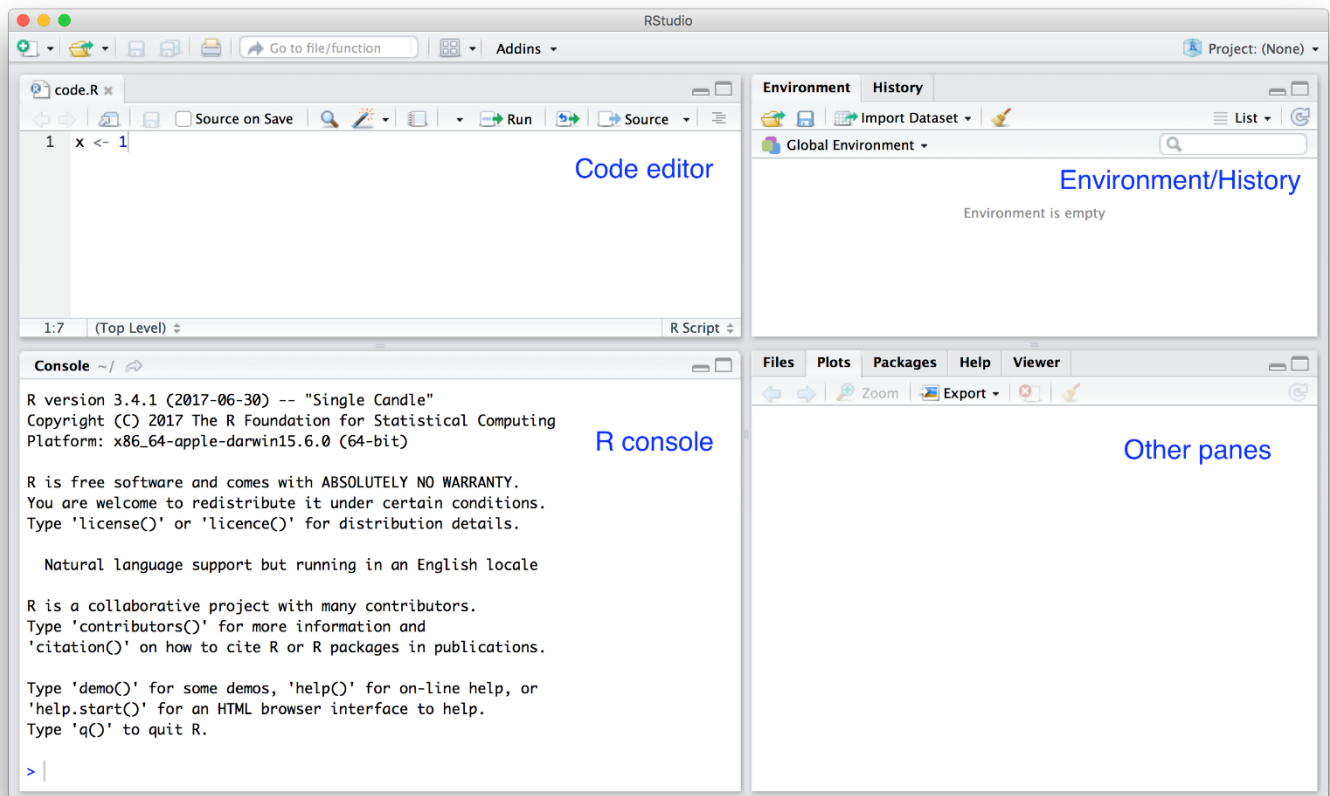
```
#> [1] 2.96
```

Note that in this book, grey boxes are used to show R code typed into the R console. The symbol `#>` is used to denote what the R console outputs.

# Scripts

One of the great advantages of R over point-and-click analysis software is that you can save your work as scripts. You can edit and save these scripts using a text editor. The material in this book was developed using the interactive *integrated development environment* (IDE) **RStudio**.

RStudio includes an editor with many R specific features, a console to execute your code, and other useful panes, including one to show figures.



Most web-based R consoles also provide a pane to edit scripts, but not all permit you to save the scripts for later use.

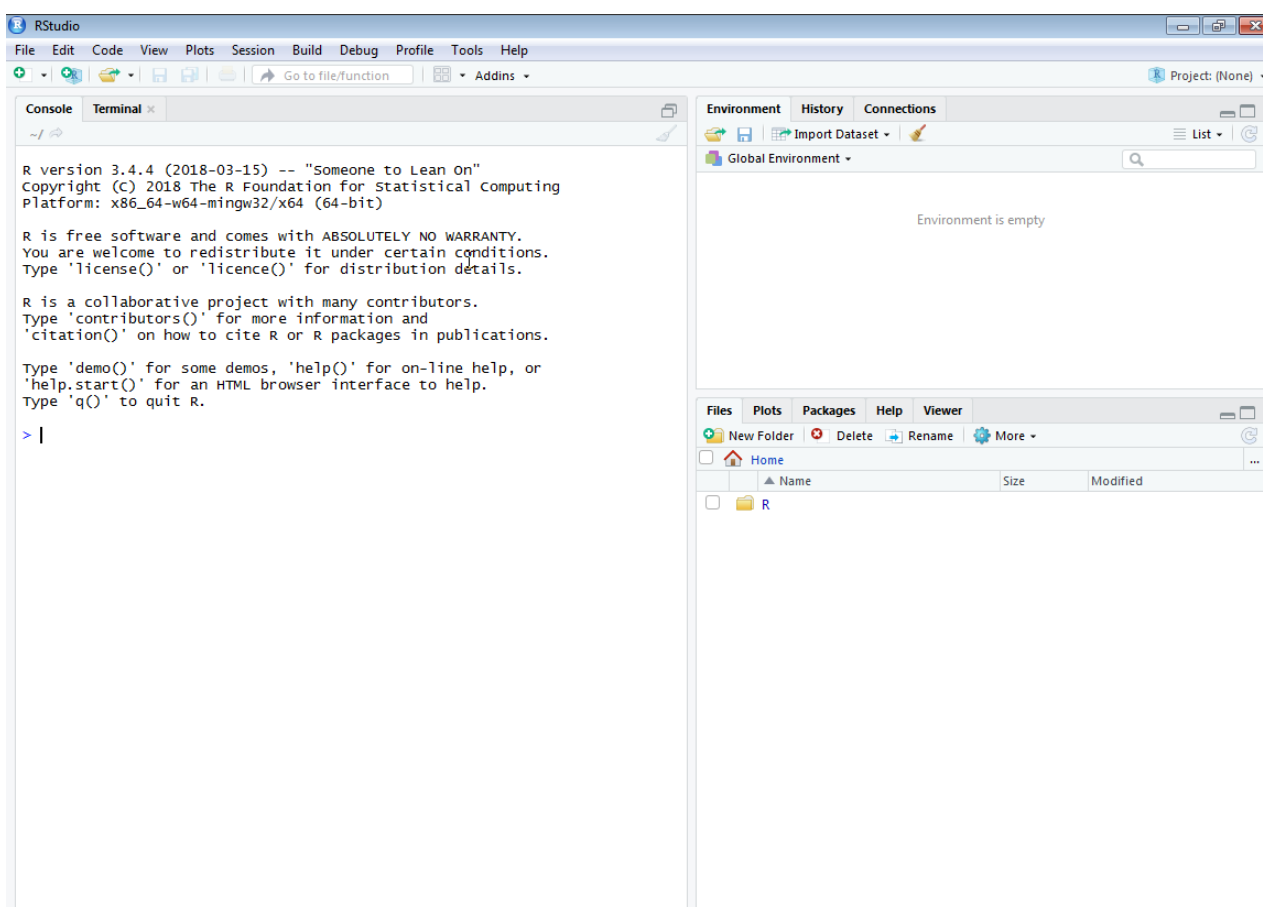
All the R scripts used to generate this book can be found on GitHub.

## RStudio

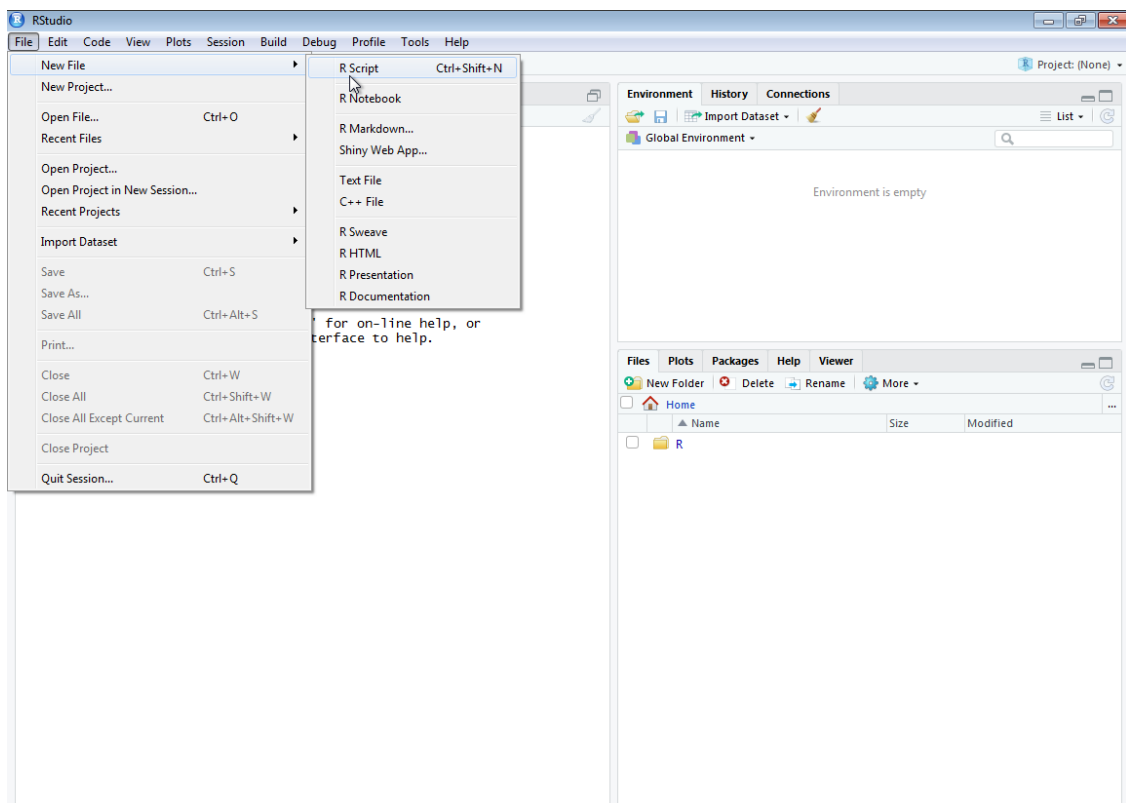
RStudio will be our launching pad for data science projects. It not only provides an editor for us to create and edit our scripts but also provides many other useful tools. In this section, we go over some of the basics.

## The panes

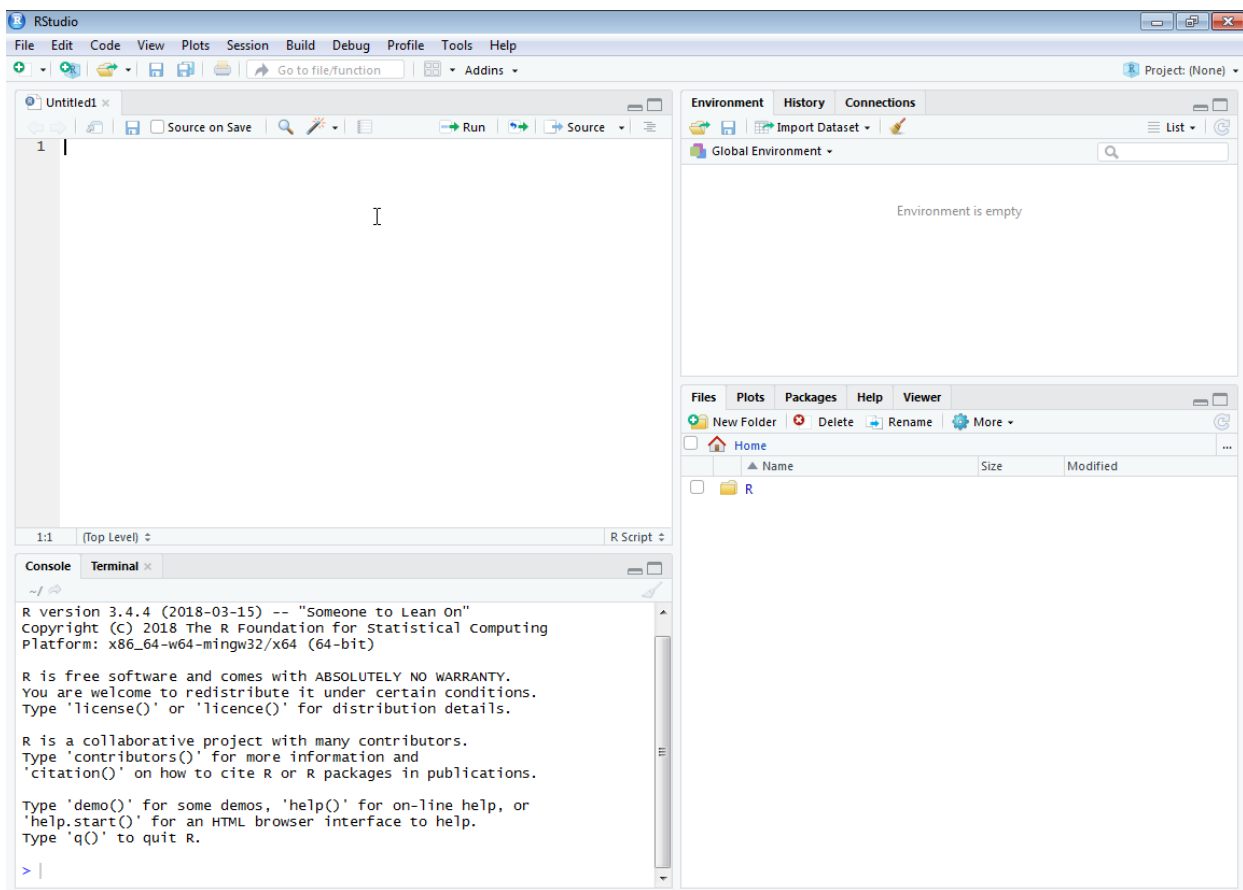
When you start RStudio for the first time, you will see three panes. The left pane shows the R console. On the right, the top pane includes tabs such as Environment and History, while the bottom pane shows five tabs: File, Plots, Packages, Help, and Viewer (these tabs may change in new versions). You can click on each tab to move across the different features.



To start a new script, you can click on File, then New File, then R Script.



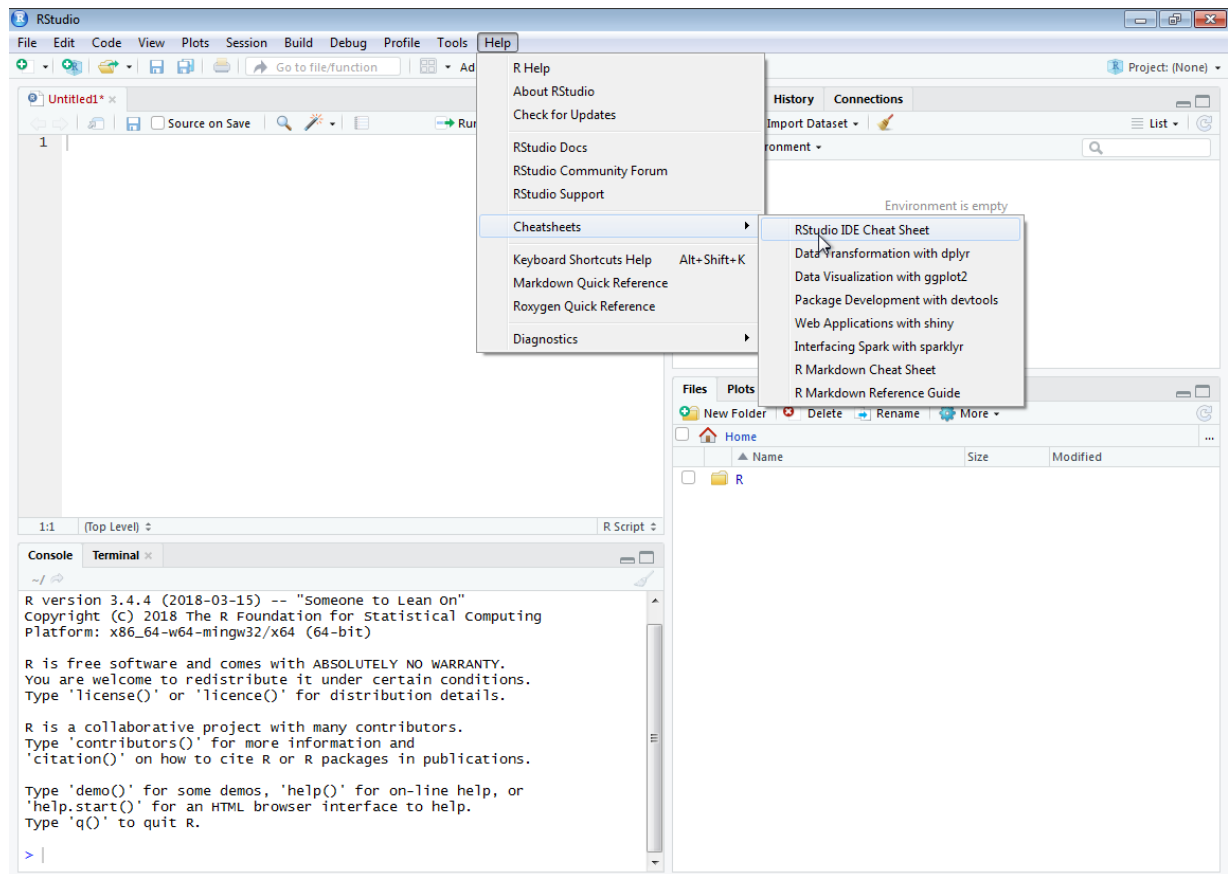
This starts a new pane on the left and it is here where you can start writing your script.



## Key bindings

Many tasks we perform with the mouse can be achieved with a combination of keystrokes instead. These keyboard versions for performing tasks are referred to as *key bindings*. For example, we just showed how to use the mouse to start a new script, but you can also use a key binding: Ctrl+Shift+N on Windows and command+shift+N on the Mac.

Although in this tutorial we often show how to use the mouse, **we highly recommend that you memorize key bindings for the operations you use most**. RStudio provides a useful cheat sheet with the most widely used commands. You can get it from RStudio directly:



You might want to keep this handy so you can look up key-bindings when you find yourself performing repetitive point-and-clicking.

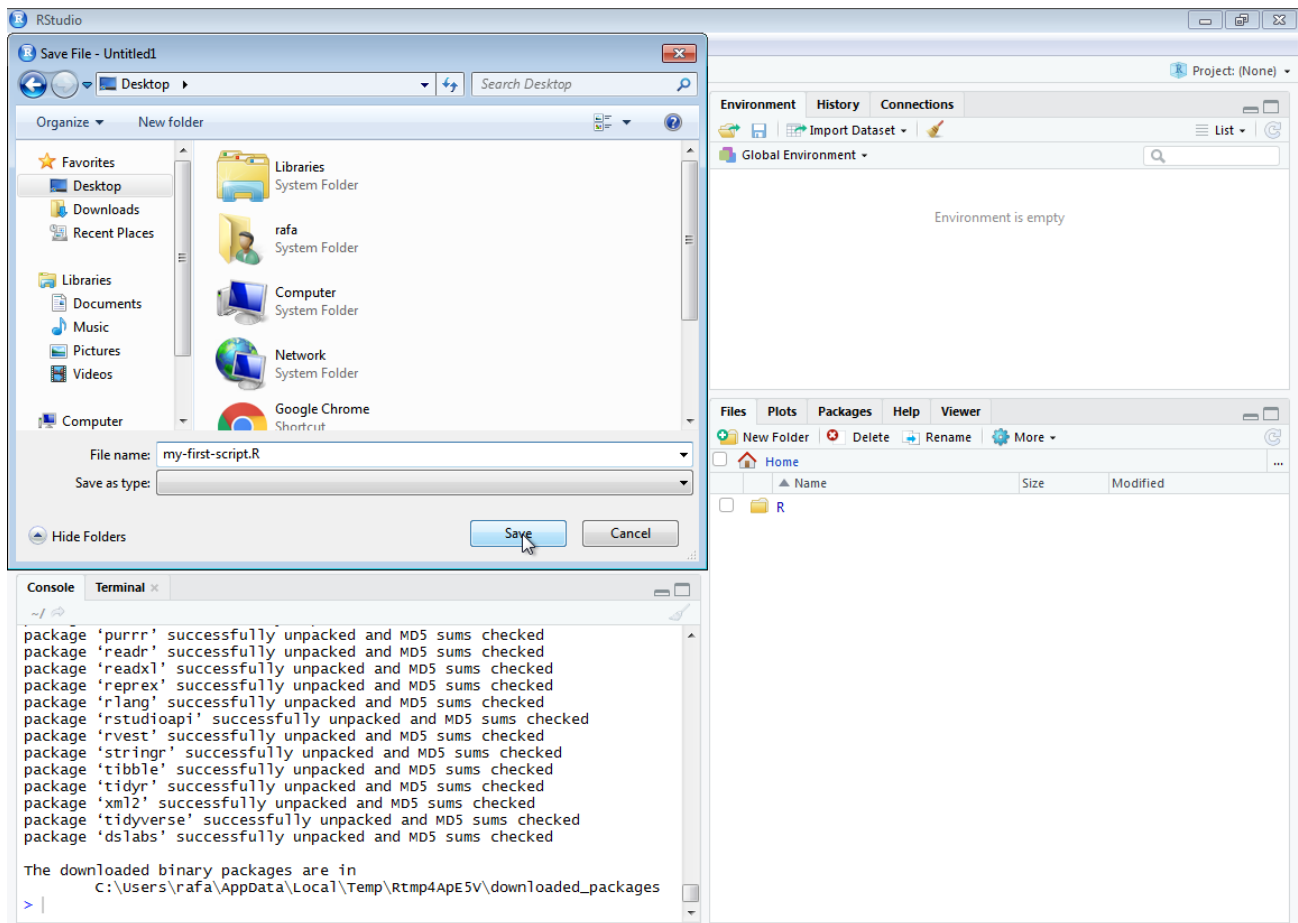
## Running commands while editing scripts

There are many editors specifically made for coding. These are useful because colour and indentation are automatically added to make code more readable. RStudio is one of these editors, and it was specifically developed for R. One of the main advantages provided by RStudio over other editors is that we can test our code easily as we edit our scripts. Below we show an example.

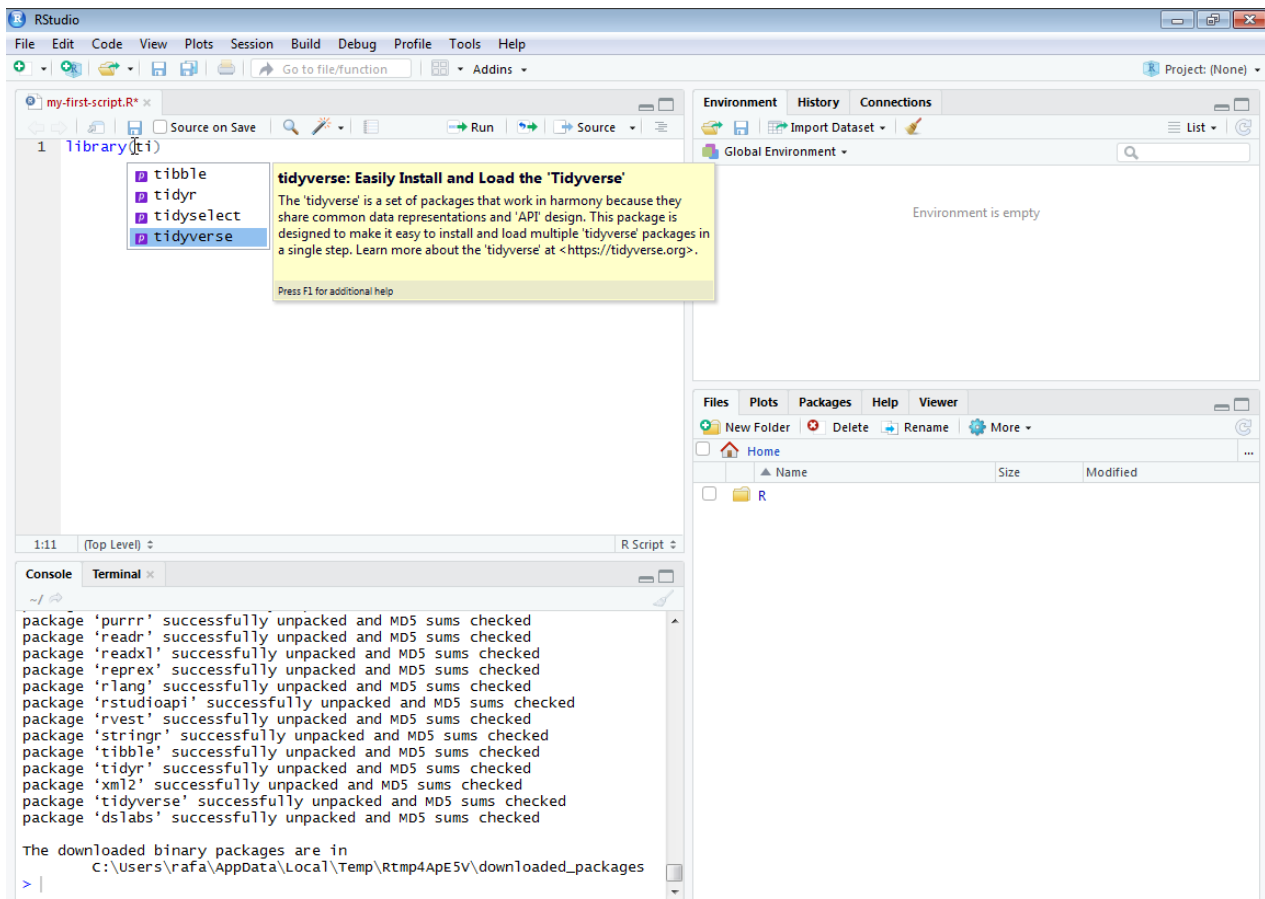
Let's start by opening a new script as we did before. A next step is to give the script a name. We can do this through the editor by saving the current new unnamed script. To do this, click on the save icon or use the key binding Ctrl+S on Windows and command+S on the Mac.

When you ask for the document to be saved for the first time, RStudio will prompt you for a name. A good convention is to use a descriptive name, with lower case letters, no spaces, only hyphens to separate words, and then followed by the suffix **.R**. We will call this script ***my-first-script.R***.





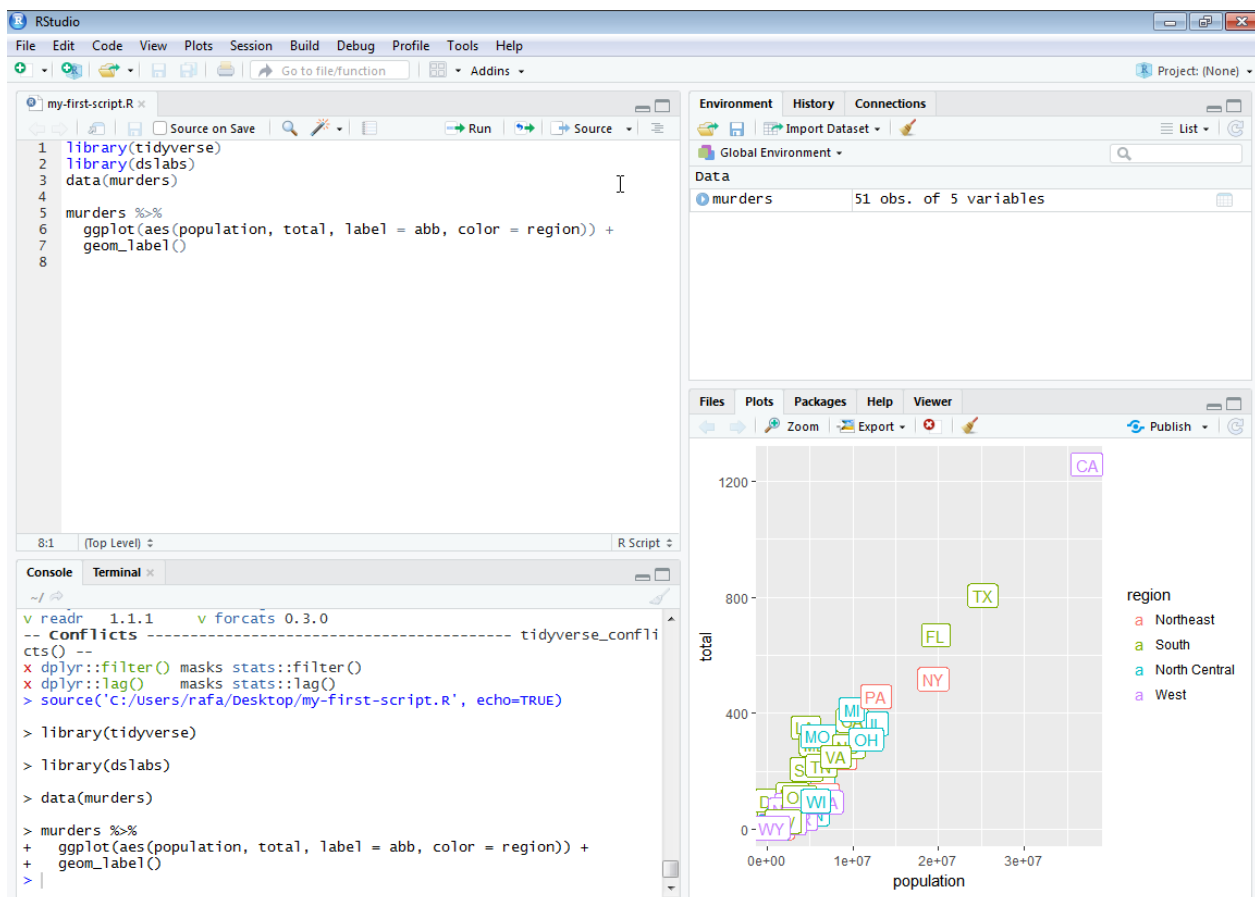
Now we are ready to start editing our first script. The first lines of code in an R script are dedicated to loading the libraries we will use. Another useful RStudio feature is that once we type **library()** it starts auto-completing with libraries that we have installed. Note what happens when we type **library(ti)**:



Another feature you may have noticed is that when you type **library(** the second parenthesis is automatically added. This will help you avoid one of the most common errors in coding: forgetting to close a parenthesis.

Now we can continue to write code. As an example, we will make a graph showing murder totals versus population totals by state. Once you are done writing the code needed to make this plot, you can try it out by executing the code. To do this, click on the Run button on the upper right side of the editing pane. You can also use the key binding: Ctrl+Shift+Enter on Windows or command+shift+return on the Mac.

Once you run the code, you will see it appear in the R console and, in this case, the generated plot appears in the plots console. Note that the plot console has a useful interface that permits you to click back and forward across different plots, zoom in to the plot, or save the plots as files.



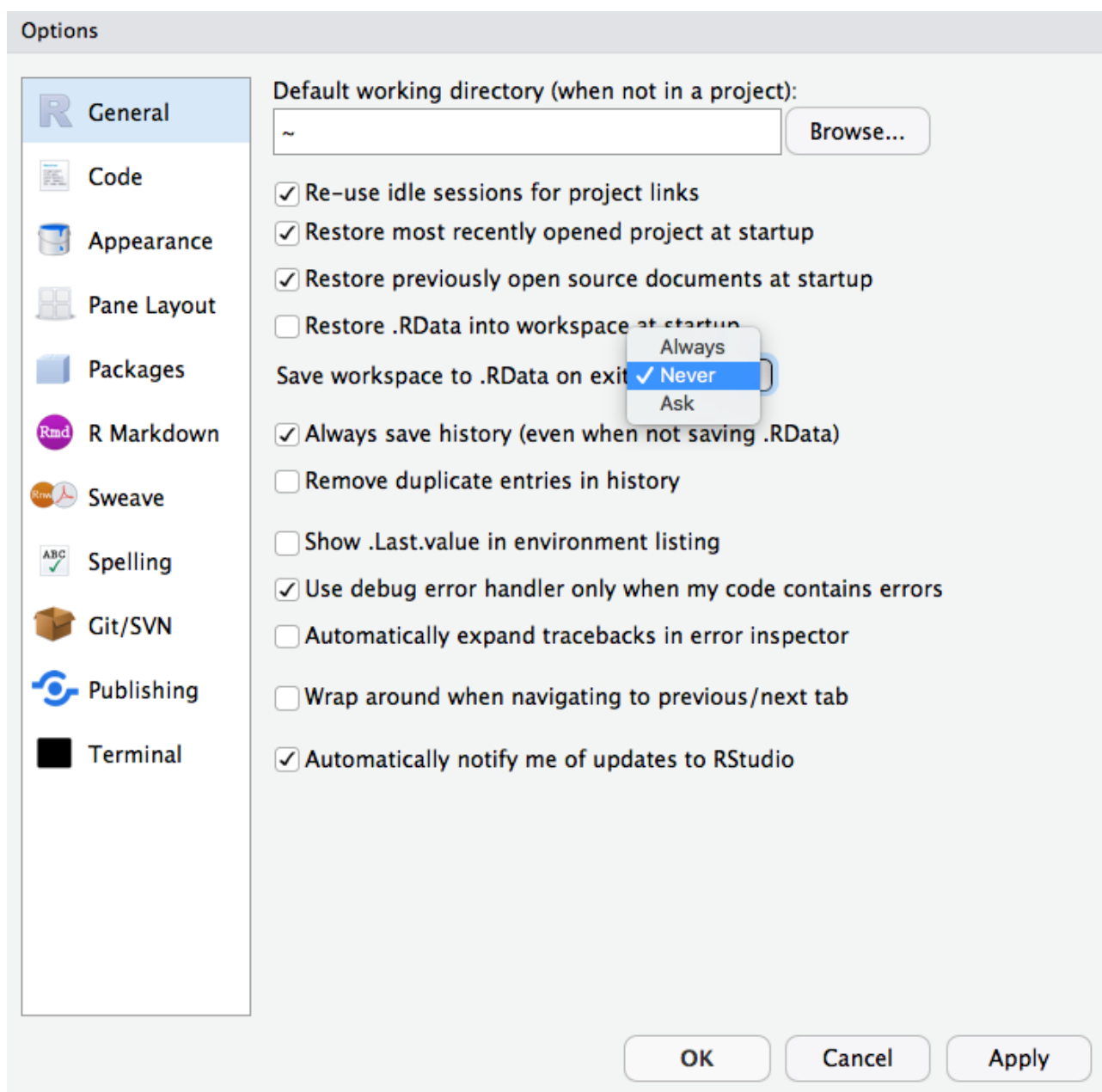
To run one line at a time instead of the entire script, you can use Control-Enter on Windows and command-return on the Mac.

## Changing global options

You can change the look and functionality of RStudio quite a bit.

To change the global options you click on Tools then Global Options....

As an example we show how to make a change that we highly recommend. This is to change the Save workspace to .RData on exit to Never and uncheck the Restore .RData into workspace at start. By default, when you exit R saves all the objects you have created into a file called .RData. This is done so that when you restart the session in the same folder, it will load these objects. We find that this causes confusion especially when we share code with colleagues and assume they have this .RData file. To change these options, make your *General* settings look like this:



## Installing R packages

The functionality provided by a fresh install of R is only a small fraction of what is possible. In fact, we refer to what you get after your first install as base R. The extra functionality comes from add-ons available from developers. There are currently hundreds of these available from CRAN and many others shared via other repositories such as GitHub. However, because not everybody needs all available functionality, R instead makes different components available via packages. R makes it very easy to install packages from within R. For example, to install the **dslabs** package, which we use to share datasets and code related to this book, you would type:

```
install.packages("dslabs")
```

In RStudio, you can navigate to the Tools tab and select install packages. We can then load the package into our R sessions using the library function:

```
library(dslabs)
```

As you go through this book, you will see that we load packages without installing them. This is because once you install a package, it remains installed and only needs to be loaded with library. The package remains loaded until we quit the R session. If you try to load a package and get an error, it probably means you need to install it first.

We can install more than one package at once by feeding a character vector to this function:

```
install.packages(c("tidyverse", "dslabs"))
```

Note that installing tidyverse actually installs several packages. This commonly occurs when a package has dependencies, or uses functions from other packages. When you load a package using library, you also load its dependencies.

Once packages are installed, you can load them into R and you do not need to install them again, unless you install a fresh version of R. Remember packages are installed in R not RStudio.

It is helpful to keep a list of all the packages you need for your work in a script because if you need to perform a fresh install of R, you can re-install all your packages by simply running a script.

You can see all the packages you have installed using the following function:

```
installed.packages()
```

For example, the following command will install plotrix package which is required for 3D charts.

```
install.packages("plotrix")
```

As a convention, we will start learning R programming by writing a "Hello, World!" program. Depending on the needs, you can program either at R command prompt or you can use an R script file to write your program. Let's check both one by one.

## R - BASIC SYNTAX

```
> myString <- "Hello, World!"  
> print (myString)  
[1] "Hello, World!"
```

Here first statement defines a string variable **myString**, where we assign a string *"Hello, World!"* and then next statement **print()** is being used to print the value stored in variable myString.

## R – DATA TYPES

Generally, while doing programming in any programming language, you need to use various variables to store various information. Variables are nothing but reserved memory locations to store values. This means that when you create a variable you reserve some space in memory.

You may like to store information of various data types like character, wide character, integer, floating point, double floating point, Boolean etc. Based on the data type of a variable, the operating system allocates memory and decides what can be stored in the reserved memory.

In contrast to other programming languages like C and java in R, the variables are not declared as some data type. The variables are assigned with R-Objects and the data type of the R-object becomes the data type of the variable. There are many types of R-objects. The frequently used ones are:

- Vectors
- Lists
- Matrices
- Arrays
- Factors
- Data Frames

The simplest of these objects is the **vector object** and there are six data types of these atomic vectors, also termed as six classes of vectors. The other R-Objects are built upon the atomic vectors.

| Data Type | Example                             | Verify                                                                                                                   |
|-----------|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Logical   | TRUE, FALSE                         | <pre>v &lt;- TRUE</pre> <pre>print(class(v))</pre> <p>it produces the following result:</p> <pre>[1] "logical"</pre>     |
| Numeric   | 12.3, 5, 999                        | <pre>v &lt;- 23.5</pre> <pre>print(class(v))</pre> <p>it produces the following result:</p> <pre>[1] "numeric"</pre>     |
| Integer   | 2L, 34L, 0L                         | <pre>v &lt;- 2L</pre> <pre>print(class(v))</pre> <p>it produces the following result:</p> <pre>[1] "integer"</pre>       |
| Complex   | 3 + 2i                              | <pre>v &lt;- 2+5i</pre> <pre>print(class(v))</pre> <p>it produces the following result:</p> <pre>[1] "complex"</pre>     |
| Character | 'a' , '"good", "TRUE", '23.4'       | <pre>v &lt;- "TRUE"</pre> <pre>print(class(v))</pre> <p>it produces the following result:</p> <pre>[1] "character"</pre> |
| Raw       | "Hello" is stored as 48 65 6c 6c 6f | <pre>v &lt;- charToRaw("Hello")</pre> <pre>print(class(v))</pre> <p>it produces the following result:</p>                |

|  |  |           |
|--|--|-----------|
|  |  | [1] "raw" |
|--|--|-----------|

In R programming, the very basic data types are the R-objects called **vectors** which hold elements of different classes as shown above. Please note in R the number of classes is not confined to only the above six types. For example, we can use many atomic vectors and create an array whose class will become array.

## Vectors

When you want to create vector with more than one element, you should use **c()** function which means to combine the elements into a vector.

```
# Create a vector.
apple <- c('red', 'green', 'yellow')
print(apple)
```

```
# Get the class of the vector.
print(class(apple))
```

When we execute the above code, it produces the following result:

```
[1] "red"      "green"    "yellow"
[1] "character"
```

## Lists

A list is an R-object which can contain many different types of elements inside it like vectors, functions and even another list inside it.

```
# Create a list.
list1 <- list(c(2,5,3), 21.3, sin)
```



```
# Print the list.  
print(list1)
```

When we execute the above code, it produces the following result:

```
[[1]]  
[1] 2 5 3
```

```
[[2]]  
[1] 21.3
```

```
[[3]]  
function (x)  .Primitive("sin")
```

## Matrices

A matrix is a two-dimensional rectangular data set. It can be created using a vector input to the `matrix` function.

```
# Create a matrix.  
M = matrix( c('a','a','b','c','b','a'), nrow = 2, ncol = 3, byrow =  
TRUE)  
print(M)
```

When we execute the above code, it produces the following result:

```
 [,1] [,2] [,3]  
[1,] "a"  "a"  "b"  
[2,] "c"  "b"  "a"
```

## Arrays

While matrices are confined to two dimensions, arrays can be of any number of dimensions. The array function takes a dim attribute which creates the required number of dimension. In the below example we create an array with two elements which are 3x3 matrices each.

```
# Create an array.  
a <- array(c('green','yellow'),dim = c(3,3,2))  
print(a)
```

When we execute the above code, it produces the following result:

```
, , 1
```

|      | [,1]     | [,2]     | [,3]     |
|------|----------|----------|----------|
| [1,] | "green"  | "yellow" | "green"  |
| [2,] | "yellow" | "green"  | "yellow" |
| [3,] | "green"  | "yellow" | "green"  |

```
, , 2
```

|      | [,1]     | [,2]     | [,3]     |
|------|----------|----------|----------|
| [1,] | "yellow" | "green"  | "yellow" |
| [2,] | "green"  | "yellow" | "green"  |
| [3,] | "yellow" | "green"  | "yellow" |

## Factors

Factors are the r-objects which are created using a vector. It stores the vector along with the distinct values of the elements in the vector as labels. The labels are always character irrespective of whether it is numeric or character or Boolean etc. in the input vector. They are useful in statistical modeling.

Factors are created using the **factor()** function. The **nlevels** functions gives the count of levels.

```
# Create a vector.
```

<-

```
apple_colors
c('green','green','yellow','red','red','red','green')

# Create a factor object.
factor_apple <- factor(apple_colors)

# Print the factor.
print(factor_apple)
print(nlevels(factor_apple))
```

When we execute the above code, it produces the following result :

```
[1] green  green  yellow red    red    red    green
Levels: green red  yellow
[1] 3
```

## Data Frames

Data frames are tabular data objects. Unlike a matrix in data frame each column can contain different modes of data. The first column can be numeric while the second column can be character and third column can be logical. It is a list of vectors of equal length.

Data Frames are created using the **data.frame()** function.

```
# Create the data frame.
BMI <- data.frame(
  gender = c("Male", "Male", "Female"),
  height = c(152, 171.5, 165),
  weight = c(81, 93, 78),
  Age = c(42, 38, 26)
)
print(BMI)
```

When we execute the above code, it produces the following result:

```
gender height weight Age
1   Male   152.0      81  42
2   Male   171.5      93  38
3 Female   165.0      78  26
```

## R – VARIABLES

A variable provides us with named storage that our programs can manipulate. A variable in R can store an atomic vector, group of atomic vectors or a combination of many R objects. A valid variable name consists of letters, numbers and the dot or underline characters. The variable name starts with a letter or the dot not followed by a number.

| Variable Name          | Validity | Reason                                                                     |
|------------------------|----------|----------------------------------------------------------------------------|
| var_name2.             | valid    | Has letters, numbers, dot and underscore                                   |
| var_name%              | Invalid  | Has the character '%'. Only dot(.) and underscore allowed.                 |
| 2var_name              | invalid  | Starts with a number                                                       |
| .var_name,<br>var.name | valid    | Can start with a dot(.) but the dot(.) should not be followed by a number. |
| .2var_name             | invalid  | The starting dot is followed by a number making it invalid.                |
| _var_name              | invalid  | Starts with _ which is not valid                                           |

### Variable Assignment

The variables can be assigned values using leftward, rightward and equal to operator. The values of the variables can be printed using **print()** or **cat()** function. The **cat()** function combines multiple items into a continuous print output.

```

# Assignment using equal operator.
var.1 = c(0,1,2,3)

# Assignment using leftward operator.
var.2 <- c("learn","R")

# Assignment using rightward operator.
c(TRUE,1) -> var.3

print(var.1)
cat ("var.1 is ", var.1 ,"\n")
cat ("var.2 is ", var.2 ,"\n")
cat ("var.3 is ", var.3 ,"\n")

```

When we execute the above code, it produces the following result:

```

[1] 0 1 2 3
var.1 is  0 1 2 3
var.2 is  learn R
var.3 is  1 1

```

**Note** – The vector `c(TRUE,1)` has a mix of logical and numeric class. So logical class is coerced to numeric class making TRUE as 1.

## Data Type of a Variable

In R, a variable itself is not declared of any data type, rather it gets the data type of the R - object assigned to it. So, R is called a dynamically typed language, which means that we can change a variable's data type of the same variable again and again when using it in a program.

```

var_x <- "Hello"
cat("The class of var_x is ",class(var_x) ,"\n")

```

```
var_x <- 34.5
cat("  Now the class of var_x is ",class(var_x), "\n")

var_x <- 27L
cat("  Next the class of var_x becomes ",class(var_x), "\n")
```

When we execute the above code, it produces the following result:

```
The class of var_x is  character
  Now the class of var_x is  numeric
    Next the class of var_x becomes  integer
```

## Finding Variables

To know all the variables currently available in the workspace we use the `ls()` function. Also the `ls()` function can use patterns to match the variable names.

```
print(ls())
```

When we execute the above code, it produces the following result:

```
[1] "my var"      "my_new_var" "my_var"      "var.1"
[5] "var.2"      "var.3"      "var.name"    "var_name2."
[9] "var_x"      "varname"
```

**Note** – It is a sample output depending on what variables are declared in your environment.

The `ls()` function can use patterns to match the variable names.

```
# List the variables starting with the pattern "var".
```

```
print(ls(pattern = "var"))
```

When we execute the above code, it produces the following result:

```
[1] "my var"      "my_new_var" "my_var"      "var.1"
[5] "var.2"      "var.3"      "var.name"    "var_name2."
[9] "var_x"      "varname"
```

The variables starting with **dot(.)** are hidden, they can be listed using "**all.names = TRUE**" argument to **ls()** function.

```
print(ls(all.name = TRUE))
```

When we execute the above code, it produces the following result:

```
[1] ".cars"          ".Random.seed" ".var_name"      ".varname"
".varname2"
[6] "my var"      "my_new_var"  "my_var"      "var.1"      "var.2"
[11] "var.3"      "var.name"    "var_name2."  "var_x"
```

## Deleting Variables

Variables can be deleted by using the **rm()** function. Below we delete the variable var.3. On printing the value of the variable error is thrown.

```
rm(var.3)
print(var.3)
```

When we execute the above code, it produces the following result:

```
[1] "var.3"
Error in print(var.3) : object 'var.3' not found
```

All the variables can be deleted by using the **rm()** and **ls()** function together.

```
rm(list = ls())  
print(ls())
```

When we execute the above code, it produces the following result:

```
character(0)
```



# R – OPERATORS

An operator is a symbol that tells the compiler to perform specific mathematical or logical manipulations. R language is rich in built-in operators and provides following types of operators.

## Types of Operators

We have the following types of operators in R programming:

- Arithmetic Operators
- Relational Operators
- Logical Operators
- Assignment Operators
- Miscellaneous Operators

## Arithmetic Operators

Following table shows the arithmetic operators supported by R language. The operators act on each element of the vector.

| Operator | Description                            | Example                                                                                                                         |
|----------|----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| +        | Adds two vectors                       | <pre>v &lt;- c( 2,5.5,6) t &lt;- c(8, 3, 4) print(v+t)</pre> it produces the following result:<br><pre>[1] 10.0  8.5 10.0</pre> |
| -        | Subtracts second vector from the first | <pre>v &lt;- c( 2,5.5,6) t &lt;- c(8, 3, 4) print(v-t)</pre> it produces the following result:                                  |

|                 |                                                               |                                                                                                                                                                                     |
|-----------------|---------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                 |                                                               | <code>[1] -6.0 2.5 2.0</code>                                                                                                                                                       |
| <code>*</code>  | Multiplies both vectors                                       | <pre> v &lt;- c( 2,5.5,6) t &lt;- c(8, 3, 4) print(v*t) </pre> <p>it produces the following result:</p> <pre> [1] 16.0 16.5 24.0 </pre>                                             |
| <code>/</code>  | Divide the first vector with the second                       | <pre> v &lt;- c( 2,5.5,6) t &lt;- c(8, 3, 4) print(v/t) </pre> <p>When we execute the above code, it produces the following result:</p> <pre> [1] 0.250000 1.833333 1.500000 </pre> |
| <code>%%</code> | Give the remainder of the first vector with the second        | <pre> v &lt;- c( 2,5.5,6) t &lt;- c(8, 3, 4) print(v%%t) </pre> <p>it produces the following result:</p> <pre> [1] 2.0 2.5 2.0 </pre>                                               |
| <code>/%</code> | The result of division of first vector with second (quotient) | <pre> v &lt;- c( 2,5.5,6) t &lt;- c(8, 3, 4) print(v/%t) </pre> <p>it produces the following result:</p> <pre> [1] 0 1 1 </pre>                                                     |
| <code>^</code>  | The first vector raised to the exponent of second vector      | <pre> v &lt;- c( 2,5.5,6) t &lt;- c(8, 3, 4) print(v^t) </pre>                                                                                                                      |

|  |  |                                                                                            |
|--|--|--------------------------------------------------------------------------------------------|
|  |  | it produces the following result:<br><br><pre>[1]      256.000      166.375 1296.000</pre> |
|--|--|--------------------------------------------------------------------------------------------|

## Relational Operators

Following table shows the relational operators supported by R language. Each element of the first vector is compared with the corresponding element of the second vector. The result of comparison is a Boolean value.

| Operator | Description                                                                                                | Example                                                                                                                                             |
|----------|------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| >        | Checks if each element of the first vector is greater than the corresponding element of the second vector. | <pre>v &lt;- c(2,5.5,6,9) t &lt;- c(8,2.5,14,9) print(v&gt;t)</pre> it produces the following result:<br><br><pre>[1] FALSE TRUE FALSE FALSE</pre>  |
| <        | Checks if each element of the first vector is less than the corresponding element of the second vector.    | <pre>v &lt;- c(2,5.5,6,9) t &lt;- c(8,2.5,14,9) print(v &lt; t)</pre> it produces the following result:<br><br><pre>[1] TRUE FALSE TRUE FALSE</pre> |
| ==       | Checks if each element of the first vector is equal to the corresponding element of the second vector.     | <pre>v &lt;- c(2,5.5,6,9) t &lt;- c(8,2.5,14,9) print(v == t)</pre>                                                                                 |

|                    |                                                                                                                               |                                                                                                                                                                                |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                    |                                                                                                                               | <p>it produces the following result:</p> <pre><b>[1] FALSE FALSE FALSE TRUE</b></pre>                                                                                          |
| <code>&lt;=</code> | <p>Checks if each element of the first vector is less than or equal to the corresponding element of the second vector.</p>    | <pre><b>v &lt;- c(2,5.5,6,9)</b> <b>t &lt;- c(8,2.5,14,9)</b> <b>print(v&lt;=t)</b></pre> <p>it produces the following result:</p> <pre><b>[1] TRUE FALSE TRUE TRUE</b></pre>  |
| <code>&gt;=</code> | <p>Checks if each element of the first vector is greater than or equal to the corresponding element of the second vector.</p> | <pre><b>v &lt;- c(2,5.5,6,9)</b> <b>t &lt;- c(8,2.5,14,9)</b> <b>print(v&gt;=t)</b></pre> <p>it produces the following result:</p> <pre><b>[1] FALSE TRUE FALSE TRUE</b></pre> |
| <code>!=</code>    | <p>Checks if each element of the first vector is unequal to the corresponding element of the second vector.</p>               | <pre><b>v &lt;- c(2,5.5,6,9)</b> <b>t &lt;- c(8,2.5,14,9)</b> <b>print(v!=t)</b></pre> <p>it produces the following result:</p> <pre><b>[1] TRUE TRUE TRUE FALSE</b></pre>     |

## Logical Operators

Following table shows the logical operators supported by R language. It is applicable only to vectors of type logical, numeric or complex. All numbers greater than 1 are considered as logical value TRUE.

Each element of the first vector is compared with the corresponding element of the second vector. The result of comparison is a Boolean value.

| Operator | Description                                                                                                                                                                                             | Example                                                                                                                                               |
|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| &        | It is called Element-wise Logical AND operator. It combines each element of the first vector with the corresponding element of the second vector and gives a output TRUE if both the elements are TRUE. | <pre>v &lt;- c(3,1,TRUE,2+3i) t &lt;- c(4,1,FALSE,2+3i) print(v&amp;t)</pre> it produces the following result:<br><pre>[1] TRUE TRUE FALSE TRUE</pre> |
|          | It is called Element-wise Logical OR operator. It combines each element of the first vector with the corresponding element of the second vector and gives a output TRUE if one the elements is TRUE.    | <pre>v &lt;- c(3,0,TRUE,2+2i) t &lt;- c(4,0,FALSE,2+3i) print(v t)</pre> it produces the following result:<br><pre>[1] TRUE FALSE TRUE TRUE</pre>     |
| !        | It is called Logical NOT operator. Takes each element of the vector and gives the opposite logical value.                                                                                               | <pre>v &lt;- c(3,0,TRUE,2+2i) print(!v)</pre> it produces the following result:<br><pre>[1] FALSE TRUE FALSE FALSE</pre>                              |

The logical operator **&&** and **||** considers only the first element of the vectors and give a vector of single element as output.

| Operator | Description                                                                                                    | Example                                                                                                                                       |
|----------|----------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| &&       | Called Logical AND operator. Takes first element of both the vectors and gives the TRUE only if both are TRUE. | <pre>v &lt;- c(3,0,TRUE,2+2i) t &lt;- c(1,3,TRUE,2+3i) print(v&amp;&amp;t)</pre> <p>it produces the following result:</p> <pre>[1] TRUE</pre> |
|          | Called Logical OR operator. Takes first element of both the vectors and gives the TRUE if one of them is TRUE. | <pre>v &lt;- c(0,0,TRUE,2+2i) t &lt;- c(0,3,TRUE,2+3i) print(v  t)</pre> <p>it produces the following result:</p> <pre>[1] FALSE</pre>        |

## Assignment Operators

These operators are used to assign values to vectors.

| Operator                   | Description            | Example                                                                                                                                                                                                                                            |
|----------------------------|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <-<br>or<br>=<br>or<br><<- | Called Left Assignment | <pre>v1 &lt;- c(3,1,TRUE,2+3i) v2 &lt;&lt;- c(3,1,TRUE,2+3i) v3 = c(3,1,TRUE,2+3i) print(v1) print(v2) print(v3)</pre> <p>it produces the following result:</p> <pre>[1] 3+0i 1+0i 1+0i 2+3i [1] 3+0i 1+0i 1+0i 2+3i [1] 3+0i 1+0i 1+0i 2+3i</pre> |

|                 |                         |                                                                                                                                                                                            |
|-----------------|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| -><br>or<br>->> | Called Right Assignment | <pre> c(3,1,TRUE,2+3i) -&gt; v1 c(3,1,TRUE,2+3i) -&gt;&gt; v2 print(v1) print(v2) </pre> it produces the following result:<br><pre> [1] 3+0i 1+0i 1+0i 2+3i [1] 3+0i 1+0i 1+0i 2+3i </pre> |
|-----------------|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Miscellaneous Operators

These operators are used to for specific purpose and not general mathematical or logical computation.

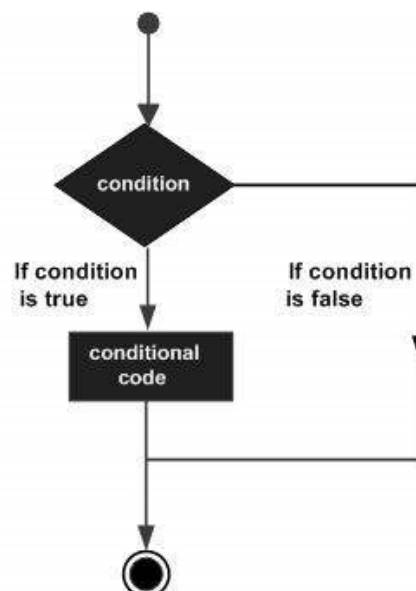
| Operator | Description                                                                | Example                                                                                                                                                 |
|----------|----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| :        | Colon operator. It creates the series of numbers in sequence for a vector. | <pre> v &lt;- 2:8 print(v) </pre> it produces the following result:<br><pre> [1] 2 3 4 5 6 7 8 </pre>                                                   |
| %in%     | This operator is used to identify if an element belongs to a vector.       | <pre> v1 &lt;- 8 v2 &lt;- 12 t &lt;- 1:10 print(v1 %in% t) print(v2 %in% t) </pre> it produces the following result:<br><pre> [1] TRUE [1] FALSE </pre> |

|    |                                                                       |                                                                                                                                                                                                          |
|----|-----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| %% | <p>This operator is used to multiply a matrix with its transpose.</p> | <pre>M = matrix( c(2,6,5,1,10,4), nrow = 2, ncol = 3, byrow = TRUE)  t = M %% t(M)  print(t)</pre> <p>it produces the following result:</p> <pre>      [,1] [,2] [1,]    65    82 [2,]    82   117</pre> |
|----|-----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## R – DECISION MAKING

Decision making structures require the programmer to specify one or more conditions to be evaluated or tested by the program, along with a statement or statements to be executed if the condition is determined to be **true**, and optionally, other statements to be executed if the condition is determined to be **false**.

Following is the general form of a typical decision-making structure found in most of the programming languages:



R provides the following types of decision-making statements. Click the following links to check their detail.

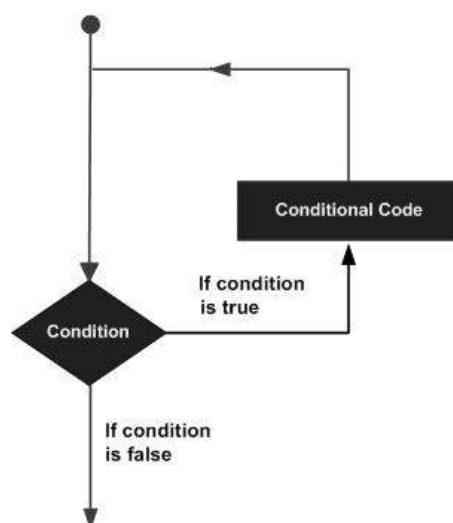


| Sr.No. | Statement & Description                                                                                                                                         |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1      | <b>if statement</b><br>An <b>if</b> statement consists of a Boolean expression followed by one or more statements.                                              |
| 2      | <b>if...else statement</b><br>An <b>if</b> statement can be followed by an optional <b>else</b> statement, which executes when the Boolean expression is false. |
| 3      | <b>switch statement</b><br>A <b>switch</b> statement allows a variable to be tested for equality against a list of values.                                      |

## R – LOOPS

There may be a situation when you need to execute a block of code several number of times. In general, statements are executed sequentially. The first statement in a function is executed first, followed by the second, and so on.

Programming languages provide various control structures that allow for more complicated execution paths.



A loop statement allows us to execute a statement or group of statements multiple times and the following is the general form of a loop statement in most of the programming languages.

R programming language provides the following kinds of loop to handle looping requirements. Click the following links to check their detail.

| Sr.No. | Loop Type & Description                                                                                                                                 |
|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1      | <b>repeat loop</b><br>Executes a sequence of statements multiple times and abbreviates the code that manages the loop variable.                         |
| 2      | <b>while loop</b><br>Repeats a statement or group of statements while a given condition is true. It tests the condition before executing the loop body. |
| 3      | <b>for loop</b><br>Like a while statement, except that it tests the condition at the end of the loop body.                                              |

## Loop Control Statements

Loop control statements change execution from its normal sequence. When execution leaves a scope, all automatic objects that were created in that scope are destroyed.

R supports the following control statements. Click the following links to check their detail.

| Sr.No. | Control Statement & Description                                                                                                         |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------|
| 1      | <b>break statement</b><br>Terminates the <b>loop</b> statement and transfers execution to the statement immediately following the loop. |
| 2      | <b>Next statement</b><br>The <b>next</b> statement simulates the behavior of R switch.                                                  |